PROVEIT / BUSY BEAVER / ZFC

A Common Macro Library and Smaller Semantic Interfaces

for the K215 Correctness Proofs

A source-grounded proposal with checked Rocq and Lean components

Prepared for Vladimir Reshetnikov

October 21, 2026 (America/Los_Angeles)

Reviewed repository revision:

`6d9800d1ca67c52db0eb6e1c65df7b5cab9cdb95`

Abstract

The most useful further reduction is to make the existing macro proofs depend on a small common program interface. The maintained Rocq libraries for the dominated and command-permuted programs have identical mathematical bodies after systematic renaming. Their 58 fixed-node contracts use only 89 node definitions. We isolate that graph, compile the entire inherited macro library against it, prove a general transport theorem for the actual register-machine semantics, bind the graph to both literal programs, and check all 58 transported K215 contracts. This provides the verified core of a shared production library.

On the Lean side, we provide a checked two-file patch sharing envelopes and bounded tagged comprehension, with a net reduction of 20 lines and 899 bytes. A separate functional-seed experiment proves a constructive normalization theorem and removes an unnecessary irreflexivity hypothesis from augmented separation. Its support code currently exceeds its local line savings, so it is presented as an optional interface improvement. The report distinguishes these component checks from the repository's existing full-endpoint checks and states the remaining integration obligations explicitly.

The archive contains this manuscript and PDF, source overlays and patches, reproducible checks, source hashes, compiler evidence, and independent source reviews.

Contents

1 Recommendation and scope **2**
1.1 The established endpoint . . . 2
1.2 Evidence levels used here . . . 2

2 What the recent commits have already simplified **3**

3 Rocq: one library over an 89-node interface **3**
3.1 A reproducible source census . . . 3
3.2 The transport theorem . . . 4
3.3 A finite gate with an explicit bound . . . 5
3.4 What was compiled . . . 5
3.5 Production integration and honest size accounting . . . 6

4 Lean: share envelopes and tagged comprehension **6**
4.1 The repeated argument . . . 6
4.2 The envelope theorem . . . 6
4.3 Changes and validation . . . 7

5 Optional Lean experiment: functional seed closure **7**
5.1 Why witnesses are repeatedly reconstructed . . . 7
5.2 A checked counterexample to omitting the hypothesis . . . 8
5.3 A stronger augmented-comprehension theorem . . . 8
5.4 Why this is an optional patch . . . 9

6 Validation, reproducibility, and integration order **9**
6.1 What the archive establishes . . . 9
6.2 Replaying the artifacts . . . 10
6.3 Recommended integration sequence . . . 10

7 Further research questions **10**

8 Assessment **11**

1 Recommendation and scope

I recommend a staged change. First, apply the small Lean envelope patch, whose mathematical content, old API statements, and affected semantic clients have been checked. Next, make Rocq's shared macro graph a production interface and replace candidate-specific copies of the arithmetic proofs with instances of that interface. Keep the functional-seed normalization as a separate proposal until its reuse or conceptual benefit justifies its extra support code.

The main architectural contribution is concrete: all 58 fixed-node macro contracts in `ProgLibPerm.v` can be obtained from a library proved over only 89 literal node definitions. This is useful evidence for a refactor because the arithmetic library was actually recompiled after replacing the full candidate program by that smaller graph. The candidate-specific transport theorems preserve all register values and all exit forms. They can therefore supply the exact kinds of contracts consumed by the next program layer.

The task concerns simplification of the correctness proof of the accepted 215-state machine. The underlying machine table is left fixed. Its dense-table SHA-256 is

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090.`

The reviewed revision is pinned so that every source observation and proposed patch has a definite meaning. The recent commits have already implemented several substantial simplifications; the following sections distinguish those existing improvements from this packet's additions.

1.1 The established endpoint

The repository records the externally checked statement

$$\mathrm{Halt}(K215, \mathrm{blank}) \quad \longleftrightarrow \quad \text{ZFC has a finite derivation of falsity.}$$

Negating this equivalence yields nonhalting if and only if syntactic consistency, without a consistency premise. The Lean endpoint is `KEndpoint215.blank_halts_iff_zfc_refutation`; the Rocq dense endpoint is `tm215_halts_iff_ZFC_inconsistent` [1, 3].

The distinction between this external proof-assistant theorem and a proof encoded in the object theory matters. The verification record explicitly leaves an internal PA proof and a derivation in ZFC's own proof syntax outside the completed result [1]. The proposed refactors preserve the existing formal target. They introduce no additional internalization claim.

1.2 Evidence levels used here

The report uses three separate levels of evidence:

1. **Repository-recorded endpoint checks.** These are the existing full or incrementally justified checks described in the maintained verification records. They establish the baseline; this work does not rerun the expensive complete machine endpoints.

2. **New component checks.** These are actual compiler runs for the new Rocq transport/library components and the changed Lean semantic modules, plus typed clients and assumption reports. The archive includes their reproduction scripts and receipts.

3. **Source and finite-data audits.** These compare literal tables, compute graph footprints, compare statements, and bind files by hash. They support the interpretation of the formal checks but do not substitute for them.

2 What the recent commits have already simplified

The current maintained Lean checkpoint is phase fourteen. It records an endpoint closure of 178 project modules and 2,093,112 bytes of selected source. The original full Lean check used 204 project modules. The original Rocq endpoint check used 237 sources. These are measurements from the repository's verification records, with their stated dependency reuse and toolchain boundaries [2, 1].

Area	Current evidence at the reviewed revision	Consequence for this proposal
Propositional and derived rules	A shared implication core, ordinary/restricted instances, and shared derived-rule proofs are already maintained.	Further duplication here needs a specific new target.
Profile composition	Six contracts now use transitive containers directly; the duplicate `KProfileTC` module and its generator were retired.	The same abstraction need not be proposed again.
AX routine	Commit `ab6ceabe32ee` makes the 217 AX proof use the checked macro-word pattern already used for the 215.	A macro-word port alone would repeat existing work.
Enumerator start state	Commit `6d9800d1ca67` generalizes the 217 enumerator and makes its seed theorem an instance.	The latest seed proof is already smaller.
Quotient development	Commit `eb931b07ad8d` preserves a one-quotient adapter patch as a proposal with its own validation scope.	It remains a relevant integration candidate, with separate evidence.

Table 1: Current work acknowledged before choosing a further simplification.

The proposals index is useful for discovery but contains historical status paragraphs. Some items in its older "Open" list were adopted by later checkpoints. For current status, the maintained sources and phase-fourteen record take precedence [2, 4].

The inherited Rocq macro library already has a sound sequence-expansion mechanism: `seqx`, `expand`, and a composition proof. It would therefore be misleading to claim that replacing per-node AX reasoning by a word interpreter is a new Rocq opportunity. The duplication that remains is at a different boundary: the same macro mathematics is forked for different surrounding program graphs.

3 Rocq: one library over an 89-node interface

3.1 A reproducible source census

The independent census parses the maintained literal Rocq tables and separately checks their agreement with the source JSON. It also strips nested comments from the two macro libraries, consistently renames candidate-specific identifiers, and normalizes whitespace. The resulting library bodies are exactly equal [7].

The 89-node set is more useful than the larger 466-node region. A common library needs the nodes actually reached from its contract roots. The extra agreeing nodes belong to other parts of the program and add no support to those contracts.

For precision, "89-node" means 89 explicitly retained node definitions at their original indices. The prototype does not renumber the nodes into an interval of length 89. A default row completes the total lookup function outside the retained set. All theorem applications are guarded by membership in the checked footprint, so that default is irrelevant to the transferred macro runs.

Measured item	Value
`ProgLibDom.v`	781 lines / 38,002 bytes
`ProgLibPerm.v`	780 lines / 37,951 bytes
Literal nodes in `DagDom` / `DagPerm`	484 / 487
Changed shared node indices	139, 141, 147, 471
Added target indices	484, 485, 486
Largest closed agreeing source region	466 nodes
Fixed-node macro statements	58
Distinct node roots in those statements	53
Descendant closure of the 53 roots	**89 nodes**

Table 2: Literal-source measurements, reproduced by `audit_macro_footprint.py`.

Proposition 3.1 (Least structural footprint)**.** *Let R be the set of 53 roots occurring in the audited fixed-node statements. Starting with R and repeatedly adding every node child of an included node terminates at an 89-element set S. It is the least forward-closed set containing R in the literal graphs.*

Proof. The finite audit enumerates the closure until no child is missing. Its final set is forward closed and contains R. Every forward-closed set containing R contains the initial set and, inductively, each set produced by one more child-expansion step. It therefore contains the final set. Agreement of the two tables on every included node makes the same closure valid for both graphs. □

This is a structural minimality statement for the chosen roots and literal edges. It is not a lower bound on a different implementation of the macros or on the number of machine states.

3.2 The transport theorem

Write $\mathrm{Run}_p(e, R, x, R')$ for the repository's actual big-step register-machine relation: program p, starting edge e, initial register file R, exit x, and final register file R'. Its exit forms are normal termination, break, continue, and halt. Break and continue carry their existing counters.

Definition 3.2. For a set of node indices S, an edge is *inside* S if it is primitive or has the form $\mathrm{node}(n)$ with $n \in S$. For programs p, q, the predicate $\mathrm{Closed}(p, q, S)$ says that every $n \in S$ satisfies at least one of:

1. $p(n) = q(n)$ and both children of this row are inside S;
2. an independently supplied contract

$$\forall R, x, R', \quad \mathrm{Run}_p(\mathrm{node}(n), R, x, R') \Rightarrow \mathrm{Run}_q(\mathrm{node}(n), R, x, R').$$

Theorem 3.3 (Local program transport)**.** *If* $\mathrm{Closed}(p, q, S)$ *and e is inside S, then*

$$\mathrm{Run}_p(e, R, x, R') \implies \mathrm{Run}_q(e, R, x, R')$$

for every initial register file, exit, and final register file.

Proof. Induct on the finite derivation of the run. A primitive constructor is independent of the program's node table and is reproduced unchanged. At a node, inspect its closure witness. If the witness is a semantic contract, apply it to the whole node run. Otherwise the two node rows are equal, and the required child edges are inside S. Apply the induction hypotheses to the child derivations and reconstruct the same node constructor in q.

The recursive continue-one constructor has three subderivations: one for the first child, one for the second child, and a resumed run of the original node. The first two use closure of the children. The third uses $n \in S$ again. Thus the argument does not assume termination of all executions and does not require a decreasing node index. It is induction on the supplied finite derivation, including its reentry subderivation. □

The theorem is checked in `LocalProgram.v`. Its design follows the existing Lean `RMTransfer` interface [5]; the contribution here is a checked Rocq counterpart with concrete source bindings. The earlier graph report in the repository had already identified the missing Rocq replacement theorem [6].

3.3 A finite gate with an explicit bound

The executable checker tests literal row equality and closure of both children at every active index below a bound. Its soundness theorem takes the additional premise

$$\forall n, \quad \mathrm{ok}(n) = \mathrm{true} \Rightarrow n < B.$$

This bound premise is essential: otherwise a finite checker could silently miss active indices outside its scan. The concrete interface uses a Boolean footprint with a built-in upper bound, and the proof of that premise is explicit.

The literal graph bindings use the first branch of Closed. The semantic-contract branch is retained for future replacements of a changed node by a proved equivalent implementation; no unproved contract is used to justify the present graph.

Since the literal conditions hold in both directions, Theorem 3.3 gives the exact equivalence

$$n \in S \implies (\mathrm{Run}_{\mathrm{Perm}}(\mathrm{node}(n), R, x, R') \leftrightarrow \mathrm{Run}_{\mathrm{Dom}}(\mathrm{node}(n), R, x, R')).$$

The root node 0 lies outside S. This theorem concerns the common macros; it does not identify the different command enumerators at their roots.

3.4 What was compiled

The checked component contains a canonical program, the inherited macro library retargeted to it, finite bindings to the two literal candidate programs, and a complete 58-contract K215 adapter. The retargeting changes the program import and the concrete program/node identifiers. The arithmetic proof bodies are preserved. The independent audit checks this exact transformation.

The adapter passes through a single lifting lemma. In schematic form:

```
canonical_sem e m exit m' -> inside macro_ok e = true ->
  concrete_sem dagPerm e m exit m'
```

This is stronger than a halting-only simulation: loop, pairing, unpairing, stack, comparison, and fetch contracts need specific exits and final registers.

The final component was compiled from fresh objects using Coq 8.18.0: 13 project modules passed, with 63 closed assumption reports, including all 58 transported contracts. Recursive `coqchk` also passed and reported no context axioms or unsafe declaration categories. The repository's maintained full Rocq endpoint was checked with Rocq 9.1.0 and rocq-stdlib 9.0.0 [1]. These are separate validation scopes. The portable component's reference inputs agree with the maintained source evidence modulo line endings; its proof semantics were not replaced by a toy model.

The K215 adapter imports only its own binding. Its ten-project-module source closure excludes the 217 candidate table and the optional cross-program comparison. This prevents a prototype convenience import from accidentally becoming a dependency of the 215 endpoint.

3.5 Production integration and honest size accounting

The eight new component files total 1,300 lines and 61,524 bytes. They include new infrastructure and one concrete adapter. The canonical arithmetic library itself remains 780 lines. These numbers demonstrate checked reuse, not an achieved reduction of the complete endpoint.

There is also a nominal-type issue: the prototype's register record is `MacroLibrary.St`, whereas the old concrete file declared its own `ProgLibPerm.St`. Equality of displayed theorem statements after relocation does not establish identity of those two record types. A production migration should share the record and its utility definitions, preserve any needed aliases, and rebuild the unchanged clients. The archive therefore does not label the Rocq component a drop-in replacement.

The intended production structure is:

1. A common register/semantics layer, parameterized by the program where necessary.
2. A canonical macro graph and one arithmetic proof library.
3. A small program-interface instance for each candidate, verified against its literal table.
4. A reusable adapter or functor exporting the contracts, with compatibility names where required.
5. Candidate-specific AX, dispatch, and root proofs above this common layer.

This boundary also stabilizes recompilation. A change confined to AX or dispatch currently changes a module imported by the macro proofs. After factoring, the canonical arithmetic library depends only on its own 89 rows. The candidate binding changes, while the common arithmetic development remains fixed. This is an architectural benefit; no measured end-to-end build-speed improvement is claimed here.

4 Lean: share envelopes and tagged comprehension

4.1 The repeated argument

In the semantic lower chain, t is an empty raw marker and the tagged domain consists of objects z satisfying $t \in z$. A raw bound U is converted into an untagged envelope, then raw tagged comprehension supplies a set whose tagged members satisfy the desired predicate.

Before the patch, `KLowerTag.dt_comp`, `KLowerTag.tag_collection`, and `KLowerQHyp.envelope` each perform the same envelope construction. `dt_comp` and `dt_comp_env` also repeat the same guarded-membership reasoning. The latter already states the more reusable contract but lives too late in the dependency graph [9].

The patch moves that existing contract to `KLowerTag` and introduces a shared envelope theorem. The namespace and explicit public statement of `dt_comp_env` are preserved; `KLowerQHyp` continues to see it through its existing transitive imports.

4.2 The envelope theorem

Assume the raw constructions supplied by `FormSeed`, irreflexivity of membership, and emptiness of t. For $a \neq t$, choose

$$E = \{a\} \cup \{z \in a : t \in z\}$$

in the raw membership sense: the equality here describes exactly which objects are members, without assuming raw extensionality.

Proposition 4.1 (A common envelope contract)**.** *There is an E with*

$$E \neq t, \qquad t \notin E, \qquad \forall z,\ (t \in z \wedge z \in a) \Rightarrow z \in E.$$

Proof. The augmented-comprehension theorem supplies the displayed membership characterization. It implies $a \in E$. If $E = t$, emptiness of t contradicts that membership, proving $E \neq t$. If $t \in E$, the characterization gives either $t = a$, contradicting $a \neq t$, or $t \in t$, contradicting irreflexivity. Finally, every tagged member of a belongs to the second part of the characterization and therefore lies in E. □

The premise $a \neq t$ is a condition of this construction. The claim does not assert that every conceivable envelope construction fails at $a = t$. The existing special case of `dt_comp` continues to handle that bound separately.

For a prescribed envelope E, the moved `dt_comp_env` theorem builds S so that

$$t \in S, \qquad \forall z\ (t \in z \Rightarrow (z \in S \leftrightarrow z \in U \wedge \theta(z))).$$

It additionally retains a raw support statement: each raw member of S is E, t, or a member of U. Tagged z cannot equal E because $t \notin E$, and cannot equal t by irreflexivity. These facts remove the two extra raw members from the interpreted domain.

4.3 Changes and validation

The two-file patch makes four changes:

1. `raw_envelope` proves the envelope contract once for $a \neq t$.
2. `dt_comp_env` moves unchanged to the earlier layer.
3. `dt_comp` and `tag_collection` use the common envelope contract.
4. The old `envelope` theorem remains a wrapper for a tagged input.

File	Old lines	New lines	Old bytes	New bytes
`KLowerTag.lean`	214	226	9,945	10,641
`KLowerQHyp.lean`	263	231	12,635	11,040
Total	477	457	22,580	21,681

Table 3: Net reduction: 20 lines and 899 bytes, including all relocated support.

The net result is small but direct: there is one proof of each envelope/comprehension contract, and three callers use it. The old API statements were checked by typed Lean clients. The affected semantic closure, including `KLowerEqv`, `KLowerQHyp`, `tag_qhyp`, and `zf_consistent_of_formSeed`, was compiled. The latter two retain the standard profile `propext`, `Classical.choice`, and `Quot.sound`. The expensive machine-specific source, bootstrap, and site certificates are outside this component check.

5 Optional Lean experiment: functional seed closure

5.1 Why witnesses are repeatedly reconstructed

The existing seed property for a binary relation Q at an input A asserts existence of a container C satisfying two conditions:

$$\forall y,\ Q(A, y) \Rightarrow \exists z,\ Q(A, z) \wedge z \in C,$$
$$\forall y \in C,\ \exists x,\ Q(x, y) \wedge \forall x' \subseteq y\ \forall y',\ Q(x', y') \Rightarrow \exists z,\ Q(x', z) \wedge z \in C.$$

Here $x' \subseteq y$ means $\forall w,\ w \in x' \Rightarrow w \in y$ in the raw structure. The requested output y or y' need not equal the output z chosen by the seed condition. For a single-valued relation, however, equality follows. The old raw constructions repeatedly perform this extraction and equality substitution [8, 9].

Definition 5.1. A relation Q is *single valued* if

$$\forall x, y, z, \quad Q(x, y) \wedge Q(x, z) \Rightarrow y = z.$$

A *seed closure* for Q, A is a container C with:

$$\begin{aligned} \text{root: } & \forall y,\ Q(A, y) \Rightarrow y \in C; \\ \text{image: } & \forall y \in C,\ \exists x,\ Q(x, y); \\ \text{step: } & \forall y \in C\ \forall x \subseteq y\ \forall z, Q(x, z) \Rightarrow z \in C. \end{aligned}$$

Theorem 5.2 (Constructive functional normalization)**.** *For a single-valued Q, the existing seed property holds if and only if there is a seed closure for Q, A.*

Proof. Suppose C witnesses the old property. Given $Q(A, y)$, its root clause supplies $Q(A, z)$ and $z \in C$. Single valuedness gives $z = y$, so $y \in C$. For the image clause, discard the closure part of the old witness. For a closure request $Q(x, z)$ below a member $y \in C$, the old clause supplies $Q(x, z')$ and $z' \in C$; single valuedness again gives $z' = z$.

Conversely, suppose the three closure clauses hold. Answer a root request $Q(A, y)$ with the same y. For a member $y \in C$, choose the input supplied by the image clause. Answer every later request $Q(x', z)$ with z itself, using the step clause. This direction does not need single valuedness. Both directions are constructive; no choice function is selected. □

The Lean theorem `seedProp_iff_closure` is checked without axioms. Its hypotheses are explicit, and its use does not strengthen an arbitrary nondeterministic seed relation.

5.2 A checked counterexample to omitting the hypothesis

Let $V = \{0, 1\}$, define membership only by $0 \in 1$, and let $Q(x, y)$ hold for every pair. The membership relation is irreflexive. The old seed property holds at $A = 0$, witnessed by $C = 1$: answer every request by 0, and use the same answer for closure requests.

A seed closure cannot exist. Its root clause would require $1 \in C$, since $Q(0, 1)$ holds, but 1 is a member of no object. Thus the single-valuedness premise cannot simply be erased. The archive includes a Boolean Lean proof of this counterexample, also with an axiom-free assumption report.

5.3 A stronger augmented-comprehension theorem

The functional formulation also exposes an unnecessary hypothesis in the old proof of augmented separation. Let A_0 be a raw singleton of a, and consider

$$Q(u, v) \quad \Longleftrightarrow \quad (u = A_0 \wedge v = a)\ \vee\ (u = \{v\} \wedge v \in a \wedge \theta(v)).$$

Both branches are single valued. If they overlap, A_0 is a singleton of both a and v, so $v = a$. This compatibility argument suffices; it does not need to rule out overlap by an irreflexivity contradiction.

Proposition 5.3. *The `FormSeed` interface alone proves existence of a raw object b with*

$$\forall z, \quad z \in b\ \leftrightarrow\ z = a\ \vee\ (z \in a \wedge \theta(z)).$$

The separate irreflexivity premise can be omitted from this theorem.

Proof. Normalize the seed relation above using Theorem 5.2. Its root places a in the container. The image clause ensures that every container member is either a or a desired member of a. For a desired member $z \in a$, take a raw singleton s_z of z. Then $s_z \subseteq a$, and $Q(s_z, z)$ holds. The closure step below a puts z in the container. The argument uses singleton uniqueness and makes no appeal to irreflexivity. □

The new `raw_aug_core` states this result. The old `raw_aug` signature, including its irreflexivity argument, remains as a compatibility wrapper. The original elementary theorem assumption profiles are checked separately so that incidental convenience tactics do not introduce a broader local dependency profile.

5.4 Why this is an optional patch

The rewritten raw file has fewer lines, although its bytes increase slightly, and a new general interface has a further cost. Including `KSeedFunctional`, the final experiment changes 202 lines to 239 lines, a net increase of 37 lines. The corresponding total grows from 9,381 to 11,217 bytes, an increase of 1,836 bytes. The exact file hashes and measurements are in the evidence manifest. This is a clearer separation of relation-specific functionality from generic closure reasoning and it strengthens one theorem; it is not presently a net source-size reduction.

For a size-focused integration, the envelope patch can be adopted independently. The functional interface is worth adopting if more downstream constructions use it, if the stronger `raw_aug_core` is useful, or if reviewers prefer its explicit root/image/step organization. Its measurements should remain separate from the 20-line envelope saving.

6 Validation, reproducibility, and integration order

6.1 What the archive establishes

Component	New validation	Remaining boundary
Rocq transport and macro graph	Actual RM semantics, finite graph bindings, inherited macro-library compilation, all 58 K215 contracts, assumption reports, recursive `coqchk`.	Production API/functor integration, maintained Rocq 9.1 replay, and complete endpoint rebuild.
Lean envelope sharing	Two-file patch; affected semantic closure; old typed interfaces; assumption checks.	Machine-specific full endpoint replay after adoption.
Lean functional seed	Constructive normalization; rewritten raw constructions; stronger augmented separation; compatibility clients; finite counterexample.	Optional adoption; total support currently adds lines.
Source audits	Literal graph census, exact source retargeting, statement comparison, import-closure review, pinned hashes.	These are source/data checks rather than independent proofs of the endpoint.

Table 4: Validation scopes are intentionally stated per component.

The combined Lean component check uses version 4.32.0 and freshly covers 65 project modules plus three audit modules. A separate envelope-only replay checks 64 project modules and one interface audit against the original raw constructions, confirming that the smaller patch can be adopted independently. All accepted compiler invocations and final source hashes are recorded. Failed development attempts are excluded from pass claims. Existing unused-section-variable warnings in inherited modules are

distinguished from errors; the evidence does not equate a successful compile with a warning-free codebase.

6.2 Replaying the artifacts

Use a checkout of the pinned ProveIt revision. The archive README gives the exact command-line interface for each supplied checker. The workflows are:

1. Run the independent Python footprint and component audits against the checkout. These read the literal maintained sources and compare their hashes and structure.
2. Run the Rocq component builder in its isolated build directory. It stages its small reference dependency closure and compiles the supplied modules serially, followed by assumption reports and `coqchk`.
3. Run the Lean overlay checker with the pinned compiler. It resolves the source import closure from the checkout, stages the overlay, compiles dependencies serially, and checks explicit old signatures and axiom reports.
4. Apply the envelope patch independently if that is the selected production change. Apply the functional-seed patch only when that separate experiment is intended.

The packet contains source and evidence, not precompiled proof objects to be trusted as opaque binaries. Generated graph data remains a proposal checked by the proof kernel. Hashes bind the actual sources, and source-only audits have their own labels.

6.3 Recommended integration sequence

First change: envelope sharing. Apply the small two-file patch, compile its reverse semantic closure, and rerun the repository's unchanged K215 endpoint clients with the current source-transitive evidence discipline. The public contract should remain fixed. This change has direct measured source savings and a narrow review surface.

Second change: common Rocq definitions. Introduce a common register record and utility layer. Preserve aliases required by downstream uses. This resolves the nominal-type distinction before replacing any concrete library. Compile the original `L2RootPerm` client against the proposed facade.

Third change: the macro interface. Move the arithmetic proofs to the canonical graph and instantiate a reusable adapter for each candidate. Keep the 215 and 217 graph bindings independent. Synchronize the candidate producers so regeneration produces the shared structure. Recheck the exact endpoint and its assumption reports before retiring the duplicate production proof bodies.

Separate decision: functional normalization. Evaluate the optional patch on proof readability and additional uses, with its full support-code cost visible. If adopted, retain `raw_aug` as the old interface and expose `raw_aug_core` for clients that need the weaker premise.

7 Further research questions

1. **Can the 58 adapters be represented once?** A Rocq functor or parameterized contract package could export the fixed contract family from one program-interface witness. The useful

acceptance test is compilation of the unchanged client, with the common definitions and old names bound precisely.

2. **How many candidates share the same macro interface?** The current census binds two literal programs. Extending it to earlier candidates would establish the actual reuse multiplier. Different node numbering should be handled by a proved relabeling map rather than textual substitution alone.

3. **Can the footprint become a maintained semantic module?** Generate a small canonical graph from an explicit macro specification, prove its contracts once, and check each larger program against that interface. This would make candidate discovery independent of arithmetic proof maintenance.

4. **Which changed nodes admit total semantic contracts?** The transport theorem already allows a contract branch. Carefully proved contracts for modified equality, pairing, or fetch routines could extend reuse beyond literal graph equality. Totality, exit behavior, and all final registers must remain explicit.

5. **Can the functional-seed interface pay for itself?** Search later raw constructions for additional single-valued relations, or provide reusable lemmas for unions of compatible functional relations. Measure the complete support and caller changes together.

6. **Which other hypotheses can be weakened by compatible overlap?** The augmented-separation proof suggests checking places where a disjointness contradiction was used merely to identify an output. A local uniqueness proof may suffice even when disjointness is unavailable.

7. **What remains in decoder primitive factoring?** The existing prefix and semantic-site proposals still need complete primitive/return coverage. A future reduction should account for all 6,176 guarded outcomes in the maintained site proof, rather than extrapolating from a single leaf [2, 4].

8. **How should internalization be packaged?** The external halting/refutation theorem and the object-theoretic formalization goal should have separate named deliverables. A future internal theorem should identify its coding, base theory, and proof translation explicitly.

8 Assessment

The new evidence supports a specific next architectural change: an 89-node common Rocq macro interface can carry the full existing macro mathematics and all 58 concrete K215 contracts. The proof of this reuse has been carried out against the real semantics and literal candidate data. Completing the production facade and checking its clients is the remaining engineering step.

The Lean envelope patch is ready as a small, separately reviewable source reduction. The functional-seed work supplies an axiom-free normalization theorem and a stronger augmented-comprehension result, with its current size cost made explicit. Together the artifacts give a practical migration path while preserving the accepted machine, its formal endpoint, and the distinction between component validation and full-endpoint certification.

References

[1] ProveIt contributors. Formal correctness of the exact 215-state machine. Pinned revision `6d9800d1ca67`, including endpoint statements, machine hashes, original toolchains, and internalization scope.

[2] ProveIt contributors. Fourteenth K215 proof-simplification checkpoint. Maintained source and evidence accounting at the reviewed revision.

[3] ProveIt contributors. KEndpoint215.lean and KEndpoint.lean. Exact machine instance and generic endpoint composition.

[4] ProveIt contributors. Proof-simplification proposals index; formal endpoint overview. Historical proposals and current checkpoint pointers.

[5] ProveIt contributors. RMTransfer.lean and L2Perm.lean. Existing Lean closed-region transport and its macro applications.

[6] ProveIt contributors. Earlier Rocq graph report. Prior identification of the missing Rocq whole-node transport theorem.

[7] ProveIt contributors; inherited mathematics and code attributed in the source to Elliot Glazer's pinned `k230-rocq` development and its cited predecessors. ProgLibPerm.v and ProgLibDom.v. The archive preserves their attribution and reference-input hashes.

[8] ProveIt contributors. KSeed222.lean and KLowerSeed.lean. Exact seed and formula-seed interfaces.

[9] ProveIt contributors. KLowerRaw.lean, KLowerTag.lean, and KLowerQHyp.lean. Baseline raw constructions, tagged comprehension, and quotient hypotheses.